



INSTITUT INFORMATIQUE SUD AVEYRON

Reclassement professionnel

Développement web back-end

Les structures de contrôle en PHP

Les boucles

Sommaire

1	Les conditions et les opérateurs de comparaison	2
1.1	Présentation des conditions	2
1.2	Présentation des opérateurs de comparaison	2
1.3	Utiliser les opérateurs de comparaison.....	2
1.4	Les conditions en PHP.....	2
1.5	Inverser la valeur logique d'une condition.....	3
1.6	Des conditions avec les opérateurs logiques	3
1.7	Revoyons nos tables de vérité	4
1.8	Les ternaires	4
2	Les boucles en PHP	5
2.1	Présentation	5
2.2	Les opérateurs d'incrément et de décrémentation.....	6
2.3	La boucle while	7
2.4	La boucle do...while	8
2.5	La boucle for	8
2.6	La boucle foreach	9
3	Inclure des fichiers dans un autre en PHP	9
3.1	Les différentes instructions	10

1 Les conditions et les opérateurs de comparaison

1.1 Présentation des conditions

Les structures de contrôle conditionnelles (ou plus simplement conditions) vont nous permettre d'exécuter différents blocs de code selon qu'une condition spécifique est vérifiée ou pas.

Les conditions sont un passage incontournable pour rendre un site dynamique puisqu'elles vont nous permettre d'exécuter différentes instructions et afficher différents résultats selon le contexte.

En PHP, les conditions sont les suivantes :

- La condition `if` (si)
- La condition `if ... else` (si ... sinon)
- La condition `if ... elseif ... else` (si ... sinon si ... sinon)

1.2 Présentation des opérateurs de comparaison

Voici les différents opérateurs de comparaison disponibles en PHP ainsi que leur signification.

Opérateur	Définition
<code>==</code>	Permet de tester l'égalité sur les valeurs
<code>===</code>	Permet de tester l'égalité en termes de valeurs et de types
<code>!= ou <></code>	Permet de tester la différence en valeurs
<code>!==</code>	Permet de tester la différence en valeurs ou en types
<code><</code>	Permet de tester si une valeur est strictement inférieure à une autre
<code>></code>	Permet de tester si une valeur est strictement supérieur à une autre
<code><=</code>	Permet de tester si une valeur est inférieure ou égale à une autre
<code>>=</code>	Permet de tester si une valeur est supérieure ou égale à une autre

Il existe un dernier opérateur un peu particulier : le `spaceship <=>`.

Voici son comportement :

- Retourne 0 si les deux opérandes sont égaux
- Retourne -1 si l'opérande de gauche est plus petit que celui de droite
- Retourne 1 si l'opérande de gauche est plus grand que celui de droite

1.3 Utiliser les opérateurs de comparaison

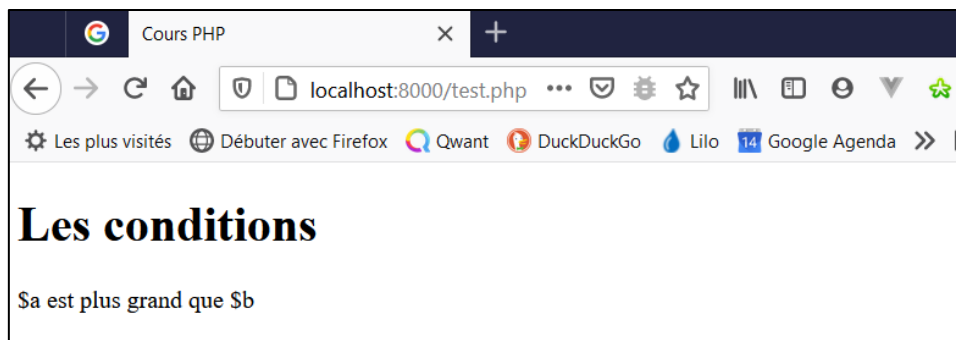
La première chose à savoir pour utiliser les opérateurs de comparaison, c'est que PHP va évaluer la comparaison de gauche à droite. Par exemple, pour `$a < $b`, on peut lire `est-ce que $a est strictement inférieur à $b`.

Si la comparaison est vérifiée, PHP renvoie la valeur booléenne `true`, dans le cas contraire, c'est `false` qui est retourné.

1.4 Les conditions en PHP

Voici un exemple d'utilisation des conditions en PHP.

```
<?php
    $a = 12;
    $b = 10;
    if($a > $b) {
        echo '$a est plus grand que $b';
    } elseif($a == $b) {
        echo '$a est egal à $b';
    } else {
        echo '$a est plus petit que $b';
    }
?>
```



Notez que les conditions `if` et `elseif` évaluent une condition vrai. Il est donc inutile et incorrect d'écrire `if( $a > $b == true )`

1.5 Inverser la valeur logique d'une condition

Nous aurons parfois besoin d'inverser la valeur logique d'une condition. Puisque PHP n'exécute le code que si la condition est vrai, nous pouvons inverser la valeur avec un `!` qui nous donnerait `if( !$a > $b )` ou écrire `if( $a > $b == false )`.

1.6 Des conditions avec les opérateurs logiques

Souvent, nous allons vouloir comparer plusieurs valeurs au sein d'une même condition, c'est-à-dire n'exécuter son code que si plusieurs conditions sont vérifiées.

Pour faire cela, nous allons pouvoir soit utiliser les opérateurs logiques, soit imbriquer plusieurs conditions les unes dans les autres.

Les opérateurs logiques vont nous permettre de créer des conditions plus puissantes mais dans certains cas il sera plus intéressant et plus rapide d'imbriquer des conditions.

Réfléchissons à ce code et à son résultat :

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Cours PHP</title>
</head>
<body>
    <h1>Les conditions et opérateurs logiques</h1>
    <?php
        $inscrit = true;
```

```

    $age = 21;

    if($inscrit) {
        if($age >= 18) {
            echo 'Utilisateur inscrit et majeur, accès autorisé';
        } else {
            echo 'Utilisateur inscrit et mineur, accès refusé';
        }
    } else {
        echo 'Utilisateur non inscrit, accès refusé';
    }
    echo '<br>';
    if($inscrit && $age >= 18) {
        echo 'Utilisateur inscrit et majeur, accès autorisé';
    } elseif($inscrit && !$age >= 18) {
        echo 'Utilisateur inscrit et mineur, accès refusé';
    } else {
        echo 'Utilisateur non inscrit, accès refusé';
    }
}
?>
</body>
</html>

```

Les deux façons d'écrire semblent donner le même résultat mais que se passe-t-il pour un `$age < 18` ?

1.7 Revoyons nos tables de vérité

Table de vérité					
a	b	a AND b a && b	A OR b a b	a XOR b	!
true	false	false	true	true	
true	true	true	true	false	
false	true	false	true	true	
false	false	false	false	false	
true					false
false					true

1.8 Les ternaires

Les structures conditionnelles ternaires (souvent simplement abrégées "ternaires") correspondent à une autre façon d'écrire nos conditions en utilisant une syntaxe basée sur l'opérateur ternaire `?:` qui est un opérateur de comparaison.

Les ternaires vont utiliser une syntaxe très condensée et nous allons ainsi pouvoir écrire toute une condition sur une ligne et accélérer la vitesse d'exécution de notre code.

Les structures ternaires vont se présenter sous la forme suivante :

`test ? code à exécuter si true : code à exécuter si false.`

```

<!DOCTYPE html>
<html lang="fr">

<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">

```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>PHP - Ternaire</title>
</head>

<body>
  <h1>Ternaires</h1>
  <?php
    $x = 4; //On affecte la valeur 4 à $x

    //Ecrire ceci :
    if($x >= 0){
      echo '$x stocke un nombre positif<br>';
    }else{
      echo '$x stocke un nombre négatif<br>';
    }

    //Est equivalent à cela :
    echo $x >= 0 ? '$x stocke un nb positif' : '$x stocke un nb négatif';
  ?>
</body>
</html>
```



Les ternaires sont d'excellentes structures conditionnelles mais elles sont à réserver à des conditions « simples » car les ternaires deviennent vite difficiles à lire.

2 Les boucles en PHP

Comme les conditions, les boucles représentent l'une des fondations de tout langage de programmation et PHP n'échappe pas à cette règle.

2.1 Présentation

Les boucles vont nous permettre d'exécuter plusieurs fois un bloc de code, c'est-à-dire d'exécuter un code « en boucle » tant qu'une condition donnée est vérifiée.

En PHP il existe quatre types de boucles :

- La boucle `while` (tant que)
- La boucle `do ... while` (faire ... tant que)
- La boucle `for` (pour)
- La boucle `foreach` (pour chaque)

Le fonctionnement général des boucles sera toujours le même : on pose une condition qui sera généralement liée à la valeur d'une variable et on exécute le code de la boucle « en boucle » tant que la condition est vérifiée.

2.2 Les opérateurs d'incrément et de décrémentation

Pour que notre condition devienne fausse à un moment, on incrémentera ou on décrémentera la valeur de notre variable à chaque nouveau passage dans la boucle.

L'incrément se représente avec `++` et la décrémentation avec `--`.

Suivant la position de l'opérateur par rapport à la variable nous pouvons exécuter une pré incrémentation `++$a` ou une post incrémentation `$a++`.

- Pré-incrément : incrémente la valeur contenue dans la variable \$a, puis retourne la valeur incrémentée.
- Post-incrément : retourne la valeur contenue dans \$a avant incrément, puis incrémente la valeur de \$a

Examinons le code suivant ainsi que le résultat dans un navigateur :

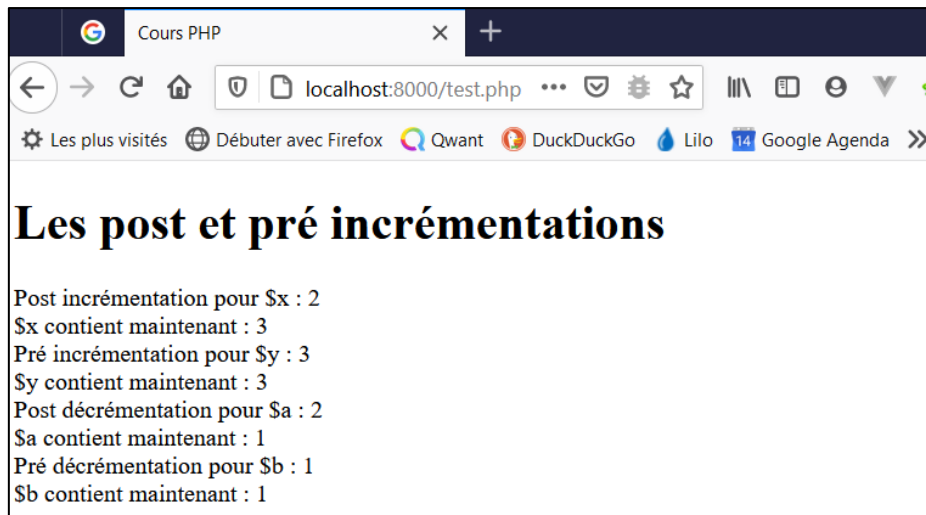
```
<?php
    $x = 2; $y = 2; $a = 2; $b = 2;

    echo 'Post incrémentation pour $x : ' . $x++. '<br>';
    echo '$x contient maintenant : ' . $x. '<br>';

    echo 'Pré incrémentation pour $y : ' . ++$y. '<br>';
    echo '$y contient maintenant : ' . $y. '<br>';

    echo 'Post décrémentation pour $a : ' . $a--. '<br>';
    echo '$a contient maintenant : ' . $a. '<br>';

    echo 'Pré décrémentation pour $b : ' . --$b. '<br>';
    echo '$b contient maintenant : ' . $b. '<br>';
?>
```



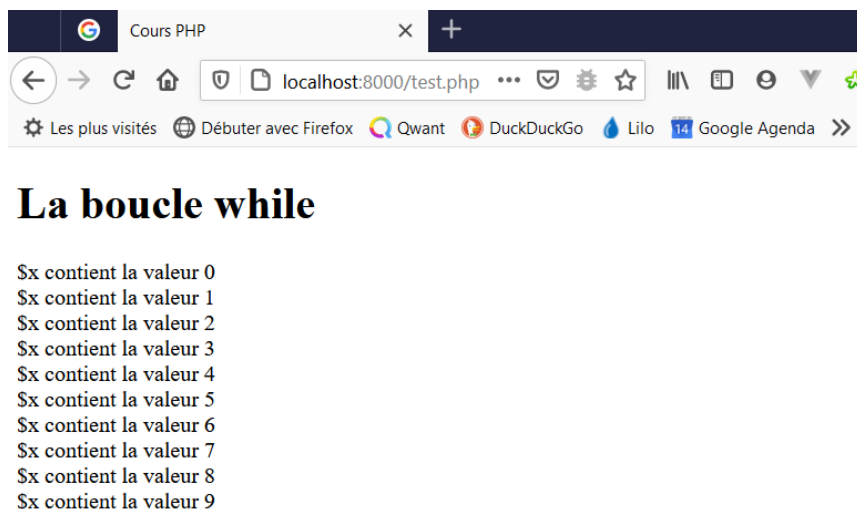
2.3 La boucle while

La boucle **while** est la boucle la plus simple à appréhender. Elle va nous permettre d'exécuter un bloc de code tant qu'une condition est vraie.

Le danger de cette boucle est de ne jamais avoir une condition fausse et de générer une boucle infinie.

Étudions le code suivant :

```
<?php
    $x = 0;
    while($x < 10) {
        echo "\$x contient la valeur $x <br>";
        $x++;
    }
?>
```



Que se passerait-il si l'incrément de **\$x** était au-dessus du **echo** ?

2.4 La boucle do...while

La boucle **do...while** va nous permettre d'exécuter du code au moins une fois même si la condition n'est pas vérifiée dès le début.

Etudions le code suivant :

```
<?php
    $x = 0;
    $y = 20;

    do{
        echo '$x contient la valeur ' . $x. '<br>';
        $x++;
    }while($x <= 3);
    echo '<br>';
    do{
        echo '$y contient la valeur ' . $y. '<br>';
        $y++;
    }while($y <= 5);
?>
```



2.5 La boucle for

Cette boucle est très puissante et c'est celle qui sera majoritairement utilisée dans nos scripts PHP.

Voici un code d'exemple :

```
<?php
    for($x = 0; $x <= 5; $x++){
        echo '$x contient la valeur ' . $x. '<br>';
    }
?>
```



2.6 La boucle foreach

La boucle `foreach` a été créée pour fonctionner avec les variables tableaux `array`.

Nous étudierons le fonctionnement de la boucle `foreach` en même temps que les tableaux `array`.

3 Inclure des fichiers dans un autre en PHP

Les instructions PHP `include` et `require` vont nous permettre toutes deux d'inclure des fichiers de code (ou plus exactement le contenu de ces fichiers) à l'intérieur d'autres fichiers de code.

Ces instructions vont nous permettre de créer des fichiers et des les inclure en fonction de nos besoins. Il est possible d'inclure un fichier n'importe où dans le code.

Test.php

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title><?php require 'title.php'; ?></title>
</head>
<body>
  <h1>Include et require</h1>

  <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Vero perspiciatis expedita minus molestiae at. Adipisci consequuntur officiis sunt rerum.</p>
  <?php require 'partial.php'; ?>
  <p>Lorem ipsum dolor sit amet consectetur adipisicing elit. Consequuntur perferendis corrupti, obcaecati quod hic dolorem facilis nostrum, aut, quibusdam cumque quisquam ab numquam doloremque dolorum pariatur eaque ipsum unde rerum. <?php require 'partial.php'; ?></p>
</body>
</html>
```

Partial.php

```
<?php  
  
echo 'Je suis dans le fichier <strong>partial.php</strong> ';
```



Ceci va être très pratique pour créer nos pages et ne pas répéter du code comme par exemple, le menu, le header, le footer ...

3.1 Les différentes instructions

La seule et unique différence entre les instructions `include` et `require` va se situer dans la réponse du PHP dans le cas où le fichier ne peut pas être inclus pour une quelconque raison (fichier introuvable, indisponible, etc.).

Dans ce cas-là, si l'inclusion a été tentée avec `include`, le PHP renverra un simple avertissement et le reste du script s'exécutera quand même tandis que si la même chose se produit avec `require`, une erreur fatale sera retournée par PHP et l'exécution du script s'arrêtera immédiatement.

L'instruction `require` est donc plus « stricte » que `include`.

La différence entre les instructions `include` et `require` et leurs variantes `include_once` et `require_once` est qu'on va pouvoir inclure plusieurs fois un même fichier avec `include` et `require` tandis qu'en utilisant `include_once` et `require_once` cela ne sera pas possible : un même fichier ne pourra être inclus qu'une seule fois dans un autre fichier.

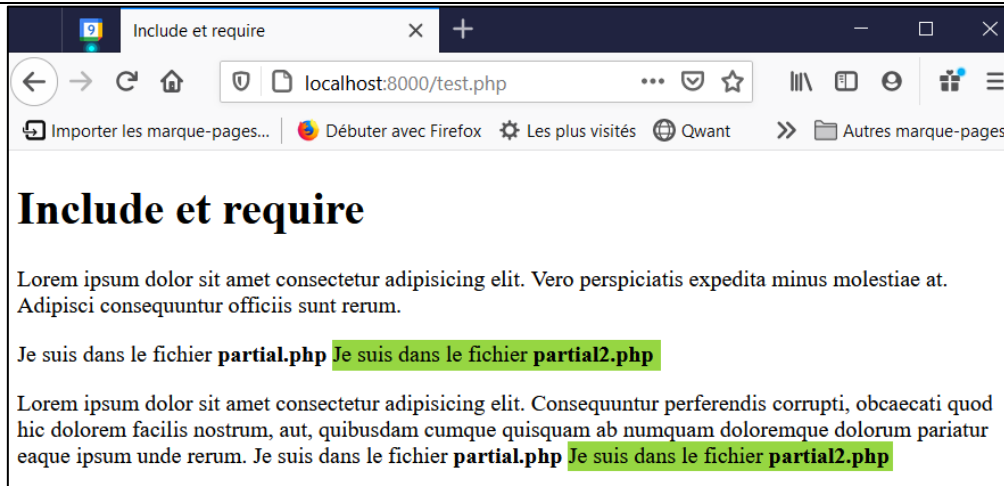
Reprenons l'exemple précédent en ajoutant un niveau d'inclusion :

Partial.php

```
<?php  
  
echo 'Je suis dans le fichier <strong>partial.php</strong> ';  
include 'partial2.php';
```

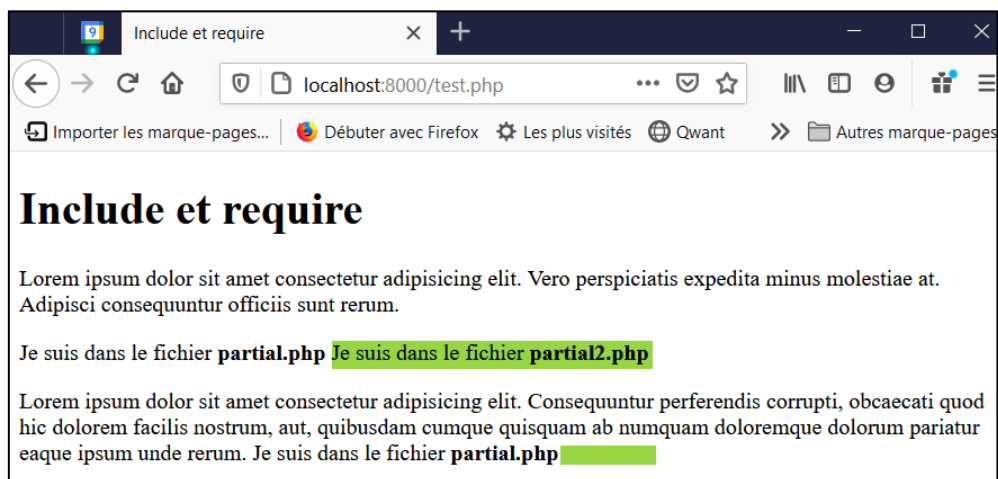
partial2.php

```
<?php  
echo 'Je suis dans le fichier <strong>partial2.php</strong>';
```



Modifions le fichier partial.php.

```
<?php  
echo 'Je suis dans le fichier <strong>partial.php</strong> '  
include_once 'partial2.php';
```



Le fichier partial2.php n'a été inclus qu'une seule fois.

Cette notion est à bien comprendre car elle peut être la source d'erreur assez difficile à détecter.